# SQL

## 一些SQL命令

```
SELECT- 从数据库中提取数据
UPDATE- 更新数据库中的数据
DELETE- 从数据库中删除数据
INSERT INTO- 将新数据插入数据库
CREATE DATABASE- 创建一个新的数据库
ALTER DATABASE- 修改数据库
CREATE TABLE- 创建新表
ALTER TABLE- 修改表格
DROP TABLE- 删除表
CREATE INDEX- 创建索引（搜索键）
DROP INDEX- 删除索引
varchar - 表示字符串
varchar() 括号内表示字符串的长度
```

[最全 SQL 字段类型（4种）、属性（6种）总结_sql字段类型-CSDN博客]

## 数据模型

### 第一部分：数据库设计的三个阶段（概念 → 逻辑 → 物理）

> **一句话概括**：概念模型是“画蓝图”，逻辑模型是“定图纸”，物理模型是“盖房子”。

#### 1. 概念模型（Conceptual Model）

- **核心任务**：搞清楚业务有哪些“核心事物”以及它们之间的业务关系。
- **面向对象**：业务人员、产品经理。完全不考虑数据库技术。
- **表现形式**：**E-R 图（实体-关系图）**。用“实体（矩形）”和“关系（菱形）”表示。
- **举例**：“用户”下“订单”。这里的“用户”和“订单”就是实体，关系是“下”。

#### 2. 逻辑模型（Logical Model）

- **核心任务**：将概念模型转化为**具体的数据库表结构**（字段、数据类型、主键、外键）。
- **面向对象**：数据库设计师。
- **关键动作**：确定每个表有哪些列，并**进行规范化（即满足范式）**。
- **举例**：将“用户”实体转为 `User` 表（id, name），将“订单”转为 `Order` 表（id, user_id, date），并用 `user_id` 外键关联起来。

#### 3. 实体模型 / 物理模型（Physical Model）

- *注：你提到的‘实体模型’在标准术语中通常指* ***物理模型（Physical Model）****，但如果是指E-R图中的‘实体’，它则属于概念层。这里按业界标准‘概念-逻辑-物理’为你讲解。*
- **核心任务**：为逻辑模型落地到**具体的物理数据库软件**（如 MySQL, Oracle）做优化。
- **面向对象**：DBA（数据库管理员）。
- **关键动作**：决定存储引擎（InnoDB/MyISAM）、建立合适的索引（Index）、考虑分区、数据压缩等性能问题。

---

# 第二部分：两大核心约束（主键 vs 外键）

这是在“逻辑模型”阶段必须定义的两把“锁”。

## 主键（Primary Key）

- **定义**：表中**唯一标识**每一行记录的字段（或字段组合）。
- **铁律**：
  1. **唯一性**：不能重复（不能有两行相同的ID）。
  2. **非空性**：绝对不能为空（NULL）。
- **举例**：`User` 表中的 `id`（用户编号）。

## 外键约束（Foreign Key）

- **定义**：用于**建立表与表之间链接**的字段。它引用另一张表的“主键”。
- **铁律**（参照完整性）：子表的外键值，必须在父表的主键中存在；否则插入/更新会被拒绝。
- **举例**：`Order` 表中的 `user_id` 必须存在于 `User` 表的 `id` 中。
- *补充*：这就是你之前问的**外键约束**，它保证了“订单必定属于一个真实存在的用户”。

---

# 第三部分：三大范式（1NF, 2NF, 3NF）

范式是**评价逻辑模型设计好坏的标准**。目的是“减少数据冗余”和“避免更新异常”。**注意**：满足第三范式（3NF）是大多数业务系统的设计基准。

## 第一范式（1NF）—— 原子性

- **要求**：表中的**所有列不可再分**（即每个单元格只能有一个值）。
- **违规示例**：`Student` 表中有一列 `Phone`，存了 `"13800138000, 13900139000"`（存了两个号码）。
- **修正**：拆成两行记录，或拆分为独立的 `StudentPhone` 子表。

## 第二范式（2NF）—— 消除部分依赖

- **前提**：必须先满足 1NF。
- **要求**：表中的**非主键列必须完全依赖于“全部主键”**。针对的是**联合主键**的情况。
- **违规示例**：成绩表 `Score` 使用联合主键 `(StudentID, CourseID)`。表中除了 `Score`（成绩），还有 `StudentName`（学生姓名）。
  - **问题**：`StudentName` 只依赖于 `StudentID`，没有依赖 `CourseID`（部分依赖），导致如果只加课程不加学生，姓名没法存，且修改姓名要改多行。
- **修正**：把 `StudentName` 移到 `Student` 表。

## 第三范式（3NF）—— 消除传递依赖

- **前提**：必须先满足 2NF。
- **要求**：表中的**非主键列之间不能有依赖关系**（即非主键列必须直接依赖于主键，不能通过其他非主键列间接依赖）。
- **违规示例**：订单表 `Order` 包含 `OrderID`（主键），`CustomerID`，`CustomerPhone`（客户电话）。
  - **问题**：`CustomerPhone` 依赖于 `CustomerID`，而 `CustomerID` 依赖于 `OrderID`。这就是传递依赖（订单 → 客户ID → 客户电话）。
- **修正**：把 `CustomerPhone` 移到 `Customer` 表，订单表只保留 `CustomerID` 外键。

## 终极串讲（一张图看懂所有概念）

假如你在设计一个电商系统：

1. **概念模型**：你和产品经理画 E-R 图，圈定有“用户”和“订单”两个实体。
2. **逻辑模型**：你开始建表，定义 `User` 表（主键 id）和 `Order` 表（主键 id，外键 user_id）。
3. **满足范式**：检查设计，确保表结构符合 **3NF**（用户名只存在用户表，订单表只存用户ID，不存冗余的用户名和电话）。
4. **物理模型**：决定这两张表在 MySQL 中用 InnoDB 引擎，给 `user_id` 加索引提升关联查询速度。
5. **约束落地**：给 `Order.user_id` 加上 **外键约束**，让数据库自动维护数据一致性。

这五个概念组合在一起，就完成了一个可靠、低冗余、高性能的数据库表结构设计。

- 关系型数据库：建立在关系模型基础上，由多张互相连接的二维表所组成的数据库
    - 使用表存储数据，格式统一，便于维护
    - 使用SQL语言操作，标准统一，使用方便

**DDL**

- 定义语言，用来定义数据库对象

**DML**

- 操作语言，用来对表中·数据进行增删改

**DQL**

- 查询语言，查询数据库中表的记录

**DCL**

- 控制语言，创建数据库用户，控制数据库访问权限

# DDL

- 查询所有数据库：`SHOW DATABASES;`
- 查询当前数据库：`SELECT DATABASE();`
- 创建：`CREATE DATABASE[IF NOT EXISTS] 数据库名[DEFAULT CHARSET字符集] [COLLATE 排序规则];`
- 删除：`DROP DATABASE[IF EXSIS]数据库名字;`
- 使用：`USE 数据库名;`
- 查询当前数据库的所有表：`SHOW TABLES;`
- 查询表结构：`DESC 表名;`
- 查询指定表的建表语句：`SHOW CREATE TABLE 表名;`
- 创建表：

```
CREATE TABLE 表名（
字段1 字段1类型 [COMMENT 表注释],
.....
字段n 字段n类型 [COMMENT 表注释]
）[COMMENT 表注释];
```

- 添加字段：`ALTER TABLE 表名ADD 字段名 类型（长度）[COMMENT 注释][约束];`

- 修改
  - 修改数据类型：`ALTER TABLE 表名MDDIFY 字段名 新数据类型（长度）`
  - 修改字段名和字段类型：`ALTER TABLE 表名 CHANGE 旧字段名 新字段名 类型（长度）[COMMENT 注释][约束];`
  - 修改表名：`ALTER TABLE 表名 RENAME TO 新表名;`
- 删除
  - 删除字段：`ALTER TABLE 表名 DROP 字段名;`
  - 删除表：`DROP TABLE[IF EXISTS] 表名;`
  - 删除指定表，并重新创建该表：`TRUNCATE TABLE 表名;`
- 数值类型

| 分类 | 类型 | 大小 | 有符号(SIGNED)范围 | 无符号(UNSIGNED)范围 | 描述 |
|---|---|---|---|---|---|
| 数值类型 | TINYINT | 1 byte | (-128, 127) | (0, 255) | 小整数值 |
| | SMALLINT | 2 bytes | (-32768, 32767) | (0, 65535) | 大整数值 |
| | MEDIUMINT | 3 bytes | (-8388608, 8388607) | (0, 16777215) | 大整数值 |
| | INT或INTEGER | 4 bytes | (-2147483648, 2147483647) | (0, 4294967295) | 大整数值 |
| | BIGINT | 8 bytes | (-2^63, 2^63-1) | (0, 2^64-1) | 极大整数值 |
| | FLOAT | 4 bytes | (-3.402823466 E+38, 3.402823466351 E+38) | 0 和 (1.175494351 E-38, 3.402823466 E+38) | 单精度浮点数值 |
| | DOUBLE | 8 bytes | (-1.7976931348623157 E+308, 1.7976931348623157 E+308) | 0 和 (2.2250738585072014 E-308, 1.7976931348623157 E+308) | 双精度浮点数值 |
| | DECIMAL | | 依赖于M(精度)和D(标度)的值 | 依赖于M(精度)和D(标度)的值 | 小数值(精确定点数) |

- 字符串类型

| 分类 | 类型 | 大小 | 描述 |
|---|---|---|---|
| 字符串类型 | CHAR | 0-255 bytes | 定长字符串 |
| | VARCHAR | 0-65535 bytes | 变长字符串 |
| | TINYBLOB | 0-255 bytes | 不超过255个字符的二进制数据 |
| | TINYTEXT | 0-255 bytes | 短文本字符串 |
| | BLOB | 0-65 535 bytes | 二进制形式的长文本数据 |
| | TEXT | 0-65 535 bytes | 长文本数据 |
| | MEDIUMBLOB | 0-16 777 215 bytes | 二进制形式的中等长度文本数据 |
| | MEDIUMTEXT | 0-16 777 215 bytes | 中等长度文本数据 |
| | LONGBLOB | 0-4 294 967 295 bytes | 二进制形式的极大文本数据 |
| | LONGTEXT | 0-4 294 967 295 bytes | 极大文本数据 |

- 日期时间类型

| 分类 | 类型 | 大小 | 范围 | 格式 | 描述 |
|---|---|---|---|---|---|
| 日期类型 | DATE | 3 | 1000-01-01 至 9999-12-31 | YYYY-MM-DD | 日期值 |
| | TIME | 3 | -838:59:59 至 838:59:59 | HH:MM:SS | 时间值或持续时间 |
| | YEAR | 1 | 1901 至 2155 | YYYY | 年份值 |
| | DATETIME | 8 | 1000-01-01 00:00:00 至 9999-12-31 23:59:59 | YYYY-MM-DD HH:MM:SS | 混合日期和时间值 |
| | TIMESTAMP | 4 | 1970-01-01 00:00:01 至 2038-01-19 03:14:07 | YYYY-MM-DD HH:MM:SS | 混合日期和时间值，时间戳 |

# 选择语句

**`SELECT` 的主要“超能力”包括：**

- **挑选指定列**：只获取你关心的信息，比如只查“姓名”和“成绩”，不查“家庭住址”等其他列。

## WHERE

用 `WHERE` 子句设定条件，比如只看“年龄大于18岁”或“订单金额大于1000元”的记录。

```
SELECT *
FROM employees
WHERE birthday > '1990-01-01';
```

- **OR，AND，NOT**

  在 SQL 的 `WHERE` 子句中，`AND`、`OR`、`NOT` 是三个最常用的逻辑运算符，用来组合多个条件，实现更灵活的过滤。

  **1. AND（与）**

  **所有条件必须同时为真**，结果行才被选中。

```
SELECT * FROM employees
WHERE department = 'Sales' AND salary > 50000;
```

  只返回“销售部”且“工资大于5万”的员工。

  **BETWEEN约等于AND**

```
SELECT * FROM products
WHERE state >= 1000 AND state =< 3000;

SELECT * FROM products
WHERE state BETWEEN 1000 AND 3000;
```

  **2. OR（或）**

  **至少有一个条件为真**，结果行就会被选中。

```
SELECT * FROM products
WHERE category = 'Electronics' OR price < 100;
```

  返回“电子类”**或者**“价格低于100”的商品（满足其一即可）。

  **IN约等于OR**

```
SELECT * FROM products
WHERE state = 'VA' OR state = 'FL' OR state = 'GA';

SELECT * FROM products
WHERE state IN ('VA','FL','GA');
```

  **3. NOT（非）**

  **取反**，即排除满足某个条件的行。

```
SELECT * FROM orders
WHERE NOT status = 'Cancelled';
```

  返回状态不是“已取消”的订单。
  也可以写成 `WHERE status != 'Cancelled'`。

  结合其他运算符：

```
SELECT * FROM users
WHERE NOT (city = 'Beijing' AND age > 30);
```

  排除“北京且年龄大于30”的用户。

  **优先级规则（从高到低）**

1. **括号** `()` —— 最高，强制先运算
2. **NOT**
3. **AND**
4. **OR** —— 最低

**示例对比：**

```
-- 不加括号：AND 先执行
SELECT * FROM student
WHERE major = 'CS' OR major = 'Math' AND grade = 'A';
-- 实际含义：major = 'CS' OR (major = 'Math' AND grade = 'A')

-- 加括号改变逻辑
SELECT * FROM student
WHERE (major = 'CS' OR major = 'Math') AND grade = 'A';
```

第一条：所有 CS 专业的学生，加上 Math 专业中成绩为 A 的学生。
第二条：只有 CS 或 Math 专业中成绩为 A 的学生。

**建议**：**不确定优先级时，一律加括号**，既清晰又避免错误。

- `LIKE` **是 SQL 中用于**模糊匹配**的操作符**，通常放在 `WHERE` 子句里，配合**通配符**来查找符合某种模式的数据。

| 通配符 | 含义 | 示例 |
| --- | --- | --- |
| `%` | 代表**任意多个字符**（包括 0 个） | `'张%'` → 以“张”开头的任何字符串 |
| `_` | 代表**恰好一个字符** | `'_李%'` → 第二个字是“李”的字符串 |

**注意**：在 SQL Server 中 `_` 也是单字符匹配；在 Access 中可能是 `?` 等，但主流数据库都是 `%` 和 `_`。

假设有 `students` 表，其中 `name` 列：

sql

```
-- 1. 查找姓“张”的人（张后面跟任意内容，包括空）
SELECT * FROM students WHERE name LIKE '张%';

-- 2. 查找名字以“伟”结尾的人
SELECT * FROM students WHERE name LIKE '%伟';

-- 3. 查找名字里包含“小”的人
SELECT * FROM students WHERE name LIKE '%小%';

-- 4. 查找三个字且姓“王”的人（'王_' 表示王+任意一个字符，共2字？不对——应该用两个下划线）
SELECT * FROM students WHERE name LIKE '王__';   -- 王 + 恰好2个字符，共3字

-- 5. 查找第二个字是“小”的人（如“张小丽”）
SELECT * FROM students WHERE name LIKE '_小%';
```

- `REGEXP`（或 `RLIKE`）**是 SQL 中更强大的**模式匹配操作符，**它使用**正则表达式**来定义匹配规则，能实现比** `LIKE` **复杂得多的模糊查询。**

  **常见正则元字符示例**

| 模式 | 含义 | 示例（匹配...） |
| --- | --- | --- |
| `^` | 开头 | `'^张'` → 以“张”开头 |
| `$` | 结尾 | `'com$'` → 以“com”结尾 |
| `.` | 任意单个字符 | `'李.'` → “李”后跟一个任意字符（如“李四”） |
| `*` | 前一个字符重复 0 或多次 | `'ba*'` → 匹配 'b', 'ba', 'baa'... |
| `+` | 前一个字符重复 1 或多次 | `'ba+'` → 匹配 'ba', 'baa'... 不匹配 'b' |
| `?` | 前一个字符重复 0 或 1 次 | `'colou?r'` → 匹配 'color' 和 'colour' |
| `[abc]` | 中括号内任意一个字符 | `'[ABC]'` → 包含 A、B 或 C |
| `[a-z]` | 范围 | `'[0-9]'` → 任意数字 |
| `[^abc]` | 否定 | `'[^0-9]'` → 非数字 |
| `\|` | 或 | `'cat\|dog'` → 包含 "cat" 或 "dog" |
| `()` | 分组 | `'(ab)+'` → "ab" 重复一次或多次 |
| `{n}` | 重复 n 次 | `'[0-9]{4}'` → 连续 4 个数字 |
| `{n,}` | 至少 n 次 | `'[a-z]{2,}'` → 至少 2 个小写字母 |
| `{n,m}` | n 到 m 次 | `'[0-9]{3,5}'` → 3-5 个数字 |

```sql
-- 1. 手机号（简单版：1开头的11位数字）
SELECT * FROM users WHERE phone REGEXP '^1[0-9]{10}$';

-- 2. 邮箱（基础匹配：包含@和点）
SELECT * FROM users WHERE email REGEXP '@.*\\.com$';

-- 3. 名字以小写字母开头，后面可以跟字母或数字
SELECT * FROM users WHERE username REGEXP '^[a-z][a-z0-9]*$';

-- 4. 包含数字
SELECT * FROM products WHERE description REGEXP '[0-9]';

-- 5. 不包含大写字母
SELECT * FROM books WHERE title NOT REGEXP '[A-Z]';

-- 6. 匹配 "cat" 或 "dog"
SELECT * FROM pets WHERE type REGEXP 'cat|dog';
```

- **查找缺失值** `IS NULL`

```sql
SELECT *
FROM customers
WHERE phone IS NULL;
```

## ORDER BY

- **排序**：用 `ORDER BY` 让结果按某个字段升序或降序排列，比如按“工资”从高到低。

  默认升序，降序在后面加上`DESC`

  ```
  SELECT *
  FROM customers
  ORDER BY firstName DESC;
  ```

## SELECT

```
-- 从名为 “employees” 的表中，查询所有数据
SELECT * FROM employees;
```

这条命令会把 `employees` 表里所有的行和列都显示出来。

如果要同时使用多个命令时候会有先后顺序，from->where->order by->limit

使用*是返回所有列，也可以输入数字等返回关键列

```
SELECT first_name, last_name FROM employees;
```

也可以再SELECT的时候进行运算

```
SELECT point * 10 + 10 AS discount_factor
FROM employees;
```

显示的数值会是原来数值*10+10,遵循运算法则，可以加括号

`AS`用于给这一列取一个别名，如果别名种有空格需要再两侧加上引号，单双都可以。‘discount factor’

`DISTINCT` 是 `SELECT` 语句中的一个关键字，用来**去掉查询结果中重复的行**，只保留唯一不同的记录。

**基本语法：**

```
SELECT DISTINCT 列1, 列2, ...
FROM 表名;
```

**举个例子：**
假设有一张 `students` 表，里面有 `name` 和 `city` 两列：

| name | city | |
|---|---|---|
| 小明 | 北京 | |
| 小红 | 上海 | |
| 小刚 | 北京 | |
| 小丽 | 深圳 | |
| 小明 | 北京 | （重复行） |

**查询所有城市**（带重复）：

```
SELECT city FROM students;
```

结果：北京, 上海, 北京, 深圳, 北京

**加上 DISTINCT 去重**：

```
SELECT DISTINCT city FROM students;
```

结果：北京, 上海, 深圳

**也可以组合多列**：`SELECT DISTINCT name, city FROM students;`
会把 (name, city) 完全相同的行合并。比如 (小明,北京) 出现两次，只保留一行。

**关键注意点：**

1. `DISTINCT` 作用于它后面**所有列的组合**，不是只跟最近的一列。
2. 它会扫描所有列的值来判断是否重复，性能上略高于普通查询，但数据量大时可能变慢。
3. 不能与 `COUNT(*)` 混在一起用错；正确用法如 `COUNT(DISTINCT city)` 来统计不重复城市的数量。

- **LIMIT**: `LIMIT` **子句用于限制** `SELECT` **语句返回的行数**，常用于分页查询或只取前几条数据

  ```
  SELECT 列 FROM 表
  ORDER BY 某列
  LIMIT [偏移量,] 行数;
  ```

# 连接

## JOIN

- 内连接：根据一个关键信息连接两张表，连接两个数据库时候类似，只需要将下方的表名换成数据库的名字。

mysql允许在表名后面直接写别名。

```
SELECT *
FROM orders o --orders的别名为o
JOIN customers
     ON o.customers_id = customers.customer_id;
```

```
USE sql_store;

SELECT *
FROM orders
JOIN customers
     ON orders.customers_id = customers.customer_id;
```

```
USE sql_inventory;

SELECT *
FROM sql_store.orders
JOIN customers
     ON orders.customers_id = customers.customer_id;
```

- 自连接：将表和自己连接

```
SELECT *
FROM orders o
JOIN orders r
     ON o.reports_to = r.customer_id;
```

- 多表连接

```
SELECT *
FROM orders o
JOIN customers
     ON o.customers_id = customers.customer_id
JOIN order_states
     ON o.customers_id = order_states.customer_id;
```

- 复合连接条件

```
SELECT *
FROM orders o
JOIN customers c
     ON o.customers_id = c.customer_id
     AND o.manager_id = c.manager.id;
```

- 隐式连接

```
SELECT *
FROM orders o, customers c
WHERE o.customers_id = c.customer_id
```

- 外连接

  内连接是并集，外连接是交集

```
SELECT *
FROM orders o --orders的别名为o
LEFT JOIN customers
     ON o.customers_id = customers.customer_id;
     --无论条件是否满足都会返回这个顾客
```

- USING

  只能在不同表中列名字完全一样的情况下去使用

```
SELECT *
FROM orders o --orders的别名为o
LEFT JOIN customers
    -- ON o.customers_id = customers.customer_id;
    USING (customers_id);--作用和上一行一样
```

- 自然连接 NATURAL JOIN

  系统进行随缘连接，结果不可靠，不建议使用

- 交叉连接 CROSS JOIN

```
SELECT *
FROM payments p
CROSS JOIN products pr;
--p和pr的每一项都会连接
```

- 联合 UNION

```
SELECT first_name, last_name
FROM employees1
UNION
SELECT first_name, last_name
FROM employees2
```

第一段查询的内容，将是联合之后所显示的列名

# 列属性

## 插入

```
INSERT INTO customers
VALUES (
    DEFAULT,-- 默认值
    'John',
    NULL -- 空
    )
```

```
INSERT INTO customers (
       last_name,
       city)
VALUES (
    'John',
    'yy')
```

- 插入多行

```
INSERT INTO products (name,quantity_in_stock,unit_product)
VALUES('aa',10,1.95),
      ('bb',11,1.95),
      ('cc',12,1.95)
```

## 创建表复制

- 把一个表复制生成另一个表

```
CREATE TABLE orders1 AS
SELECT *FROM orders2
```

! 这样复制生成的表没有主键，和自动递增

主键：唯一可以标识这个表的值（例如：学号，身份的值）

- 复制一个表的一部分到另一个表

```
INSERT INTO orders2
SELECT *
FROM orders1
WHERE birthday < '2000-01-01'
```

## 更新

- 更新单行

```
UPDATE invoices
SET
     payment_total = invoice_total * 0.5,
     payment_date = due_date
WHERE invoice_id = 1;
```

- 更新多行

```
UPDATE invoices
SET
     payment_total = invoice_total * 0.5,
     payment_date = due_date
WHERE invoice_id IN ( 1, 2);
```

- 子查询

只查询一个并且更新

```
UPDATE invoices
SET
     payment_total = invoice_total * 0.5,
     payment_date = due_date
WHERE invoice_id =
    (SELECT invoices_id FROM invoices
    WHERE name = 'aa');
```

查询多个并且更新

```
UPDATE invoices
SET
     payment_total = invoice_total * 0.5,
     payment_date = due_date
WHERE invoice_id in
    (SELECT invoices_id FROM invoices
    WHERE name in ('aa', 'bb'));
```

## 删除

- 删除

```sql
DELETE FROM orders
WHERE client_id =
        (SELECT client_id
        FROM orders
        WHERE name = 'aa')
```

# 聚合函数

它只会计算非空值，如果列中有空值将不会被计算

```sql
SELECT
    MAX(invoice_total) AS highest,
    MIN(invoice_total) AS lowest,
    AVG(invoice_total) AS average,
    SUM(invoice_total) AS total,
    COUNT(invoice_total) AS number_total_1,
    COUNT(*) AS number_total_2 #这个包含了为null的列
FROM invoices;
```

## GROUP BY

**“GROUP BY”** 在 MySQL 中的核心意思就是**“按某一列（或多列）进行分组”**。

你可以把它通俗地理解为**“数据归类”**——它将数据库中拥有相同值的行聚合在一起，通常**必须配合“聚合函数”（如 COUNT、SUM、AVG、MAX、MIN）**来使用，目的是对每一组进行统计计算，而不是单纯地罗列数据。

```sql
SELECT
    class,              -- 分组依据
    COUNT(*) AS 人数, -- 统计每班有多少人
    AVG(score) AS 平均分 -- 计算每班的平均分
FROM score
GROUP BY class;
```

使用 `GROUP BY` 时，有一条铁律：**SELECT 后面出现的非聚合字段，必须出现在 GROUP BY 后面。**

- **正确**：`SELECT class, COUNT(*) ... GROUP BY class`（class 在 GROUP BY 里）
- **错误**：`SELECT name, COUNT(*) ... GROUP BY class`（name 不在 GROUP BY 里）

> 因为分组后，每组里可能有多个 `name`（比如一班有小明和小红），数据库不知道你想显示哪一个，所以会报错或返回随机值（除非开启严格模式）。

1. **WHERE**：在**分组前**过滤原始数据（对“行”操作）。
2. **GROUP BY**：对过滤后的数据进行分组。
3. **HAVING**：在**分组后**过滤分组结果（对“组”操作）。

```sql
SELECT class, COUNT(*) AS 人数
FROM score
WHERE score > 60   -- 1. 先把不及格的踢出去
GROUP BY class     -- 2. 按班级分组
HAVING COUNT(*) >= 2; -- 3. 只留下人数>=2的班级
```

分组后默认是按分组字段排序的（MySQL 早期版本），但**不保证**顺序。如果你想要按统计结果排序，必须显式加上 `ORDER BY`：

```sql
SELECT class, SUM(score) AS 总分
FROM score
GROUP BY class
ORDER BY 总分 DESC; -- 按总分从高到低排序
```

## HAVING

在分组行之后进行筛选数据

```sql
SELECT class, SUM(score) AS 总分
FROM score
GROUP BY class
HAVING total_sales > 500
```

## ROLLUP

只能用于聚合列，可以获得对该分组的汇总值

`ROLLUP` 是 `GROUP BY` 的一个**扩展选项**，它的作用是在分组结果的基础上，**自动增加“小计”和“总计”行。**

```sql
SELECT
    date_num,
    payment_method,
    SUM(payment_num) AS total_payments,
    GROUPING(payment_method) AS is_method_summary, -- 1表示这一行是支付方式维度的小计
    GROUPING(date_num) AS is_date_summary          -- 1表示这一行是日期维度的总计
FROM payments p
JOIN paa pa ON p.aa = pa.aa
GROUP BY date_num, payment_method WITH ROLLUP;
```

`ROLLUP` 会按照你 `GROUP BY` 里字段的**顺序**，从右向左逐步“去掉”维度进行汇总：

- **第1层（最细）**：按 `(date_num, payment_method)` 分组（原始明细）。
- **第2层（小计）**：按 `(date_num)` 分组（去掉 payment_method），汇总每天总额。
- **第3层（总计）**：按 `()` 分组（全部去掉），汇总所有总额。

| date_num | payment_method | total_payments | 备注 |
| --- | --- | --- | --- |
| 2026-06-26 | 微信支付 | 100 | 明细分组 |
| 2026-06-26 | 支付宝 | 200 | 明细分组 |
| **2026-06-26** | **NULL** | **300** | **👉 日期小计（6月26日总金额）** |
| 2026-06-25 | 现金 | 50 | 明细分组 |
| **2026-06-25** | **NULL** | **50** | **👉 日期小计（6月25日总金额）** |
| **NULL** | **NULL** | **350** | **👉 总计（所有日期总和）** |

# 查询

# 子查询

```
SELECT *
FROM products
WHERE PRICE >
(   SELECT price
    FROM products
    WHERE product_id = 3)
```

- 使用IN来子查询

```
SELECT *
FROM products
WHERE product_id NOT IN
   (SELECT DISTINCT product_id
    FROM order_items)
```

DISTINCT:只显示不重复的列

- 使用EXISTS来子查询

```
SELECT *
FROM products p
WHERE EXISTS
(   SELECT product_id
    FROM order_items o
    WHERE p.id = o.id
)
```

- SELECT中使用子查询

```
SELECT
    invoice_id,
    invoice_total,
    (SELECT AVG(invoice_total)
        From invoices) AS invoice_avg,
    invoice_total - (SELECT invoice_avg)
FROM invoices
```

- FROM中的子查询

```
SELECT * FROM
(
SELECT
    invoice_id,
    name,
    (SELECT SUM(invoice_total)
        FROM invoices
        WHERE id = i.id) AS total_sales,
    (SELECT AVG(invoice_total)
        From invoices) AS invoice_avg,
    (SELECT total_sales - invoice_avg) AS differnt
FROM invoices i
)AS newquery
```

必须加上别名

## ALL

```
SELECT *
FROM invoices
WHERE invoice_total > ALL
(
    SELECT invoice_total
    FROM invoices
    WHERE client_id = 3
)
# 大于ALL(110，120，13091，329......) 大于括号里的所有值
```

## ANY

```
SELECT *
FROM invoices
WHERE invoice_total > ANY
(
    SELECT invoice_total
    FROM invoices
    WHERE client_id = 3
)
# 大于ANY(110，120，13091，329......) 大于括号里的任意值
```

# 相关子查询

```
SELECT *
FROM employees e
WHERE salary >
(
    SELECT AVG(salary)
    FROM employees
    WHERE office_id = e.office_id
)
```

这个写法的核心作用是**将子查询与外部查询的当前行“关联”起来**，让它变成一个**相关子查询。**

如果你去掉 `WHERE office_id = e.office_id`，子查询计算的是**全公司**的平均薪资；而加上它，子查询计算的是**当前员工所在办公室**的平均薪资。

具体区别如下：

**1. 逻辑目的不同**

- **不加关联条件**：查询的是“薪资高于全公司平均水平的员工”。
- **加上关联条件**：查询的是“薪资高于**自己办公室内部**平均水平的员工”（即每个办公室的“尖子生”）。

**2. 执行顺序不同（关键）**

- **非相关子查询**（不加）：数据库**只执行一次**子查询，算出全局平均值，然后拿这个固定值去比较所有员工。
- **相关子查询**（加上）：数据库**逐行扫描**外部表 `employees e`。每扫描到一名员工，就拿着该员工的 `office_id` 去子查询里跑一遍，算出该办公室的平均工资，然后再比较这名员工的工资是否高

于这个动态值。

## 数值函数

```
SELECT ROUND(5.72); #四舍五入数字--6
SELECT ROUND(5.7245, 3); #保留三位小数--5.725
SELECT TRUNCATE(5.7245, 3); #截断三位--5.724
SELECT CELING(5.7245); #返回大于或等于这个数字的整数--6
SELECT FLOOR(5.7245); #返回小于或等于这个数字的整数--5
SELECT ABS(- 5.7245); #计算绝对值--5.7245
SELECT RAND(); #生成0-1之间的随机数
```

## 字符串函数

| 函数 | 用途 | 示例 |
| --- | --- | --- |
| `CONCAT(str1, str2, ...)` | 拼接多个字符串（任一为NULL则结果为NULL） | `CONCAT('张', '三')` → `'张三'` |
| `REPLACE(str, old, new)` | 替换字符串中所有指定的子串 | `REPLACE('138-0000','-','')` → `'1380000'` |
| `TRIM([BOTH/LEADING/TRAILING] char FROM str)` | 去除字符串两端（或指定方向）的空格或指定字符 | `TRIM(' SQL ')` → `'SQL'` |
| `UPPER(str)` / `LOWER(str)` | 将字符串全部转为大写或小写 | `UPPER('abc')` → `'ABC'` |
| `CHAR_LENGTH(str)` | 返回字符串的字符个数（中英文、表情符号均按1个计算） | `CHAR_LENGTH('深圳')` → `2` |
| `SUBSTRING(str, pos, len)` / `MID()` | 从指定位置开始截取指定长度的子串（位置从1开始） | `SUBSTRING('1990-01-01', 1, 4)` → `'1990'` |
| `CONCAT_WS(separator, str1, str2, ...)` | 带分隔符拼接字符串，自动跳过NULL值 | `CONCAT_WS('-', '2026', NULL, '27')` → `'2026-27'` |
| `LEFT(str, n)` / `RIGHT(str, n)` | 从左侧或右侧截取指定个数的字符 | `RIGHT('13812345678', 4)` → `'5678'` |
| `GROUP_CONCAT([DISTINCT] expr ORDER BY expr SEPARATOR sep)` | 将多行数据的指定列聚合成一行字符串（行转列） | `GROUP_CONCAT(name SEPARATOR ';')` → `'张三;李四'` |
| `LPAD(str, len, pad)` / `RPAD(str, len, pad)` | 在字符串左侧或右侧填充指定字符至目标长度 | `LPAD('123', 5, '0')` → `'00123'` |
| `SUBSTRING_INDEX(str, delim, count)` | 按分隔符截取子串（count为正取左，负取右） | `SUBSTRING_INDEX('a.b.c', '.', 2)` → `'a.b'` |
| `INSTR(str, substr)` / `LOCATE(substr, str)` | 返回子串首次出现的位置（找不到返回0） | `INSTR('Hello', 'e')` → `2` |
| `LENGTH(str)` | 返回字符串的字节长度（UTF-8下中文占3字节） | `LENGTH('深圳')` → `6` |
| `INSERT(str, pos, len, newstr)` | 从指定位置开始，将指定长度的字符替换为新字符串 | `INSERT('abcd', 2, 2, 'xx')` → `'axxd'` |
| `REPEAT(str, count)` | 将字符串重复指定次数 | `REPEAT('*', 3)` → `'***'` |

| 函数 | 用途 | 示例 |
| --- | --- | --- |
| `REVERSE(str)` | 反转字符串 | `REVERSE('abc')` → `'cba'` |

# 日期函数

| 函数 | 用途 | 示例 |
| --- | --- | --- |
| `NOW()` | 返回当前日期和时间 | `NOW()` → `'2026-06-27 14:30:00'` |
| `CURDATE()` | 返回当前日期（不含时间） | `CURDATE()` → `'2026-06-27'` |
| `CURTIME()` | 返回当前时间（不含日期） | `CURTIME()` → `'14:30:00'` |
| `DATE_FORMAT(date, format)` | 按指定格式格式化日期 | `DATE_FORMAT(NOW(), '%Y年%m月%d日')` → `'2026年06月27日'` |
| `DATEDIFF(date1, date2)` | 返回两个日期之间相差的天数（date1 - date2） | `DATEDIFF('2026-06-30', '2026-06-27')` → `3` |
| `TIMESTAMPDIFF(unit, begin, end)` | 返回两个日期时间按指定单位计算的差值（end - begin），unit 支持 SECOND/MINUTE/HOUR/DAY/MONTH/YEAR | `TIMESTAMPDIFF(MONTH, '2026-01-01', '2026-06-27')` → `5` |
| `DATE_ADD(date, INTERVAL n unit)` | 在日期上增加指定时间间隔 | `DATE_ADD(NOW(), INTERVAL 3 DAY)` → `'2026-06-30 14:30:00'` |
| `DATE_SUB(date, INTERVAL n unit)` | 在日期上减去指定时间间隔 | `DATE_SUB(NOW(), INTERVAL 1 MONTH)` → `'2026-05-27 14:30:00'` |
| `YEAR(date)` | 提取年份 | `YEAR('2026-06-27')` → `2026` |
| `MONTH(date)` | 提取月份（1-12） | `MONTH('2026-06-27')` → `6` |
| `DAY(date)` | 提取日期中的天数（几号） | `DAY('2026-06-27')` → `27` |
| `HOUR(time)` | 提取小时 | `HOUR('14:30:00')` → `14` |
| `MINUTE(time)` | 提取分钟 | `MINUTE('14:30:00')` → `30` |
| `SECOND(time)` | 提取秒数 | `SECOND('14:30:05')` → `5` |
| `STR_TO_DATE(str, format)` | 将字符串按指定格式解析为日期 | `STR_TO_DATE('27/06/2026', '%d/%m/%Y')` → `'2026-06-27'` |
| `DATE(date)` | 提取日期时间中的日期部分 | `DATE('2026-06-27 14:30:00')` → `'2026-06-27'` |
| `TIME(datetime)` | 提取日期时间中的时间部分 | `TIME('2026-06-27 14:30:00')` → `'14:30:00'` |
| `LAST_DAY(date)` | 返回该日期所在月份的最后一天 | `LAST_DAY('2026-06-27')` → `'2026-06-30'` |
| `WEEKDAY(date)` | 返回星期几对应的数字（周一=0，周日=6） | `WEEKDAY('2026-06-27')` → `5`（周六） |
| `DAYOFWEEK(date)` | 返回星期几对应的数字（周日=1，周六=7） | `DAYOFWEEK('2026-06-27')` → `7`（周六） |
| `UNIX_TIMESTAMP(date)` | 将日期转为 Unix 时间戳（秒数） | `UNIX_TIMESTAMP(NOW())` → `...` |
| `FROM_UNIXTIME(timestamp)` | 将 Unix 时间戳转为日期时间 | `FROM_UNIXTIME(1719484200)` → `'2026-06-27 14:30:00'` |
| `DATE_ADD()` 的简便写法 `+ INTERVAL` | 直接在日期上加减时间间隔的简便语法 | `DATE_ADD('2026-06-27', INTERVAL 1 DAY)` → `'2026-06-28'` |

`DATE_ADD()` 可以换成 month / year。

## IFNULL和COALESCE

```
SELECT
    order_id,
    IFNULL(shipper_id, 'Not assigned')AS shipper #如果shipper_id为null会显示Not
assigned而不是null
FROM orders
```

```
SELECT
    order_id,
    COALESCE(shipper_id, comments, 'Not assigned')AS shipper
    #如果shipper_id为null会显示coments的内容，
    #而如果comments也是null则会显示Not assigned
FROM orders
```

## IF

`IF(condition, true_val, false_val)` 类似编程中的三元运算符。条件真返回 true_val，假返回 false_val

`IF(salary > 10000, '高薪', '普通')` → 工资过万标记为'高薪'

```
SELECT
    order_id,
    order_date,
    IF
    (
        YEAR(order_date) = YEAR(NOW()),
        'ACTIVE',
        'ARCHIVED'
    )
FROM orders
```

## CASE

```
SELECT
    order_id,
    order_date,
    CASE
        WHEN YEAR(order_date) = YEAR(NOW()) THEN 'ACTIVE'
        WHEN YEAR(order_date) = YEAR(NOW()) - 1 THEN 'ARCHIVED'
        ELSE 'FUTURE'
    END
FROM orders
```

# 创建视图

```
CREATE VIEW sales_by AS #创建一个视图模板如下
SELECT
    order_id,
    order_date,
    CASE
        WHEN YEAR(order_date) = YEAR(NOW()) THEN 'ACTIVE'
        WHEN YEAR(order_date) = YEAR(NOW()) - 1 THEN 'ARCHIVED'
        ELSE 'FUTURE'
    END
FROM orders
```

```
SELECT *
FROM sales_by
```

两个结果一样

## 更改或删除

```
CREATE OR REPLACE VIEW sales_by AS
SELECT
    order_id,
    order_date,
    CASE
        WHEN YEAR(order_date) = YEAR(NOW()) THEN 'ACTIVE'
        WHEN YEAR(order_date) = YEAR(NOW()) - 1 THEN 'ARCHIVED'
        ELSE 'FUTURE'
    END
FROM orders
```

删除

```
DROP VIEW sales_by;
```

更新

```
UPDATE sales_by
SET due_date = DATE_ADD(due_date, INTERVAL 2 day)
```

使用`WITH CHECK OPTION`来防止在更新或者删除视图时候，一些其他的行被删除

# 存储过程

## 创建一个存储过程

```
DELIMITER $$
CREATE PROCEDURE get_clients()
BEGIN
    SELECT * FROM clients;
END$$

DELIMITER ;
```

## 删除存储过程

```
DROP PROCEDURE IF EXISTS get_clients
```

```
DROP PROCEDURE get_client
```

## 参数

```
DELIMITER $$
CREATE PROCEDURE getClient
(
    name char(22)
)
BEGIN
    SELECT * FROM clients c
    WHERE c.name = name;
END $$
DELIMITER ;

CALL getClient('nnnn');
```

- 带默认值的参数

```
DELIMITER $$
CREATE PROCEDURE getClient
(
    name char(22)
)
BEGIN
    IF name IS NULL THEN
        SET name='11';
    END IF;
    SELECT * FROM clients c
    WHERE c.name = name;
END $$
DELIMITER ;

CALL getClient('nnnn');
```

```
DELIMITER $$
CREATE PROCEDURE getClient
(
    name char(22)
)
BEGIN
    SELECT * FROM clients c
    WHERE c.name = IFNULL(name, c.name);#类似三目运算符
END $$
DELIMITER ;

CALL getClient('nnnn');
```

# DECLARE

在 MySQL 的存储过程、函数和触发器中，`DECLARE` 是用来声明**局部变量**、**游标**和**条件处理程序**的关键字。

它的核心使用规则只有一条，但极其严格：`DECLARE` **必须作为** `BEGIN...END` **块中的第一条语句，且必须集中声明在所有非声明语句（如** `SET`、`SELECT`、`IF` **等）之前。**

**1. 声明局部变量（最常用）**

用于存储临时数据，可以指定数据类型和默认值。

```
-- 语法
DECLARE 变量名 数据类型 [DEFAULT 默认值];

-- 实际例子
BEGIN
    -- 声明一个整数变量，默认值为 0
    DECLARE v_count INT DEFAULT 0;

    -- 声明一个字符串变量，默认为空字符串
    DECLARE v_name VARCHAR(100) DEFAULT '';

    -- 声明一个日期变量，默认为当前日期（注意：DEFAULT 不支持函数时，要用 SET 赋值）
    DECLARE v_today DATE;

    -- 执行语句给变量赋值
    SET v_count = 100;
    SELECT user_name INTO v_name FROM users WHERE id = 1;
END
```

**关键点**：如果不写 `DEFAULT`，初始值就是 `NULL`。

**2. 声明游标（遍历多行数据）**

游标用于遍历查询结果集中的每一行记录。**注意**：游标必须在**变量声明之后**。

```
BEGIN
    -- 1. 先声明变量（用来接收游标取出的数据）
    DECLARE v_id INT;
    DECLARE v_name VARCHAR(100);

    -- 2. 再声明游标（绑定一个查询结果集）
    DECLARE cur_user CURSOR FOR SELECT id, name FROM users WHERE status = 1;

    -- 3. 最后声明处理程序（后面会讲）

    OPEN cur_user; -- 打开游标
    FETCH cur_user INTO v_id, v_name; -- 取第一行
    CLOSE cur_user; -- 关闭游标
END
```

**3. 声明条件处理程序（错误捕获）**

这是 `DECLARE` 最复杂也最强大的用法，用于定义发生异常或找不到数据时怎么办。**切记：处理程序必须声明在变量和游标之后！**

```
BEGIN
    DECLARE v_id INT DEFAULT 1;
    DECLARE v_name VARCHAR(100);
    DECLARE cur_user CURSOR FOR SELECT name FROM users;

    -- 声明处理程序：当游标遍历完所有行（02000 状态）时，设置标志为 1
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    -- 声明处理程序：发生 SQL 异常时，回滚并输出错误信息
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT '发生严重错误，事务已回滚' AS ErrorMsg;
    END;
END
```

**三种处理类型**：

- `CONTINUE`：捕获异常后，继续执行后面的代码。
- `EXIT`：捕获异常后，立即退出当前的 `BEGIN...END` 块。
- `UNDO`（不支持）：MySQL 暂不支持。

# 触发器

- **触发时机**：`BEFORE`（操作之前）或 `AFTER`（操作之后）。
- **触发事件**：`INSERT`、`UPDATE` 或 `DELETE`。
- **触发主体**：关联的哪张表。
- **触发频率**：MySQL 触发器是 `FOR EACH ROW`（行级触发），即操作影响几行数据，触发器就执行几次。
- **INSERT（插入）**：只有 `NEW`（代表新插入的行），没有 `OLD`。
- **DELETE（删除）**：只有 `OLD`（代表被删除的行），没有 `NEW`。
- **UPDATE（更新）**：既有 `OLD`（更新前的旧值），又有 `NEW`（更新后的新值）

```
CREATE TRIGGER 触发器名称
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON 表名
FOR EACH ROW
BEGIN
    -- 触发器核心逻辑代码
END;
```

- **查看所有触发器**：`SHOW TRIGGERS;`
- **查看特定库的触发器**：`SHOW TRIGGERS FROM 数据库名;`
- **删除触发器**：`DROP TRIGGER 触发器名称;`
- **修改触发器**：MySQL 不支持 `ALTER TRIGGER`，只能先 `DROP` 再 `CREATE`。

```
DELIMITER //
CREATE TRIGGER trg_check_salary
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = '薪水必须大于0! ';
    END IF;
END//
DELIMITER ;
```

# 事务

**事务** 是一组 SQL 语句组成的**逻辑处理单元**。这组语句要么**全部执行成功**（提交），要么**全部执行失败**（回滚），绝不允许出现"执行了一半"的情况。

- **原子性 (Atomicity)**：事务是一个不可分割的整体。比如转账，扣钱和加钱必须同时成功；如果中间断了，整个操作会回滚，钱不会凭空消失。
- **一致性 (Consistency)**：事务执行前后，数据库的完整性约束没有被破坏。比如转账后，总金额不变。
- **隔离性 (Isolation)**：多个事务并发执行时，彼此互不干扰。一个事务未提交的数据，对其他事务通常是不可见的（具体程度取决于隔离级别）。
- **持久性 (Durability)**：一旦事务提交，数据的修改就会永久保存到硬盘上，即使断电也不会丢失。

```
-- 1. 开启事务（两种方式）
START TRANSACTION;  -- 推荐
-- 或
BEGIN;

-- 2. 执行你的 SQL 操作（比如更新、插入、删除）
UPDATE accounts SET balance = balance - 100 WHERE user_id = 1;
UPDATE accounts SET balance = balance + 100 WHERE user_id = 2;

-- 3. 判断结果，选择结束方式
-- 如果都成功了，就提交（持久化）
COMMIT;

-- 如果中间出错了，就回滚（撤销所有修改）
ROLLBACK;
```

当多个事务同时操作同一份数据时，会出现脏读、不可重复读、幻读等问题。MySQL（InnoDB）通过隔离级别来平衡性能与一致性：

- **READ UNCOMMITTED（读未提交）**：最低级别，可能读到未提交的脏数据（**脏读**），基本不用。
- **READ COMMITTED（读已提交）**：只能读到已提交的数据（解决脏读），但有**不可重复读**问题（同一查询两次结果不同）。Oracle默认级别。
- **REPEATABLE READ（可重复读）**：MySQL 的**默认级别**。保证同一事务内多次读取结果一致（解决不可重复读），但存在**幻读**（InnoDB 通过间隙锁 MVCC 极大程度上解决了它）。
- **SERIALIZABLE（串行化）**：最高的隔离级别，事务逐个执行，性能最低，安全性最高。

查看当前隔离级别：`SELECT @@TRANSACTION_ISOLATION;`

# 索引

- **创建索引**

```
EXPLAIN SELECT customer_id FROM customers WHERE state = 'CA';

CREATE INDEX idx_state ON customers (state);
```

用于加速搜索

- 查看索引

`SHOW INDEXES IN customers`

- 前缀索引

`CREATE INDEX idx_state ON customers (lastName(20));`

## LIKE

`LIKE` 是 SQL 中用于**模糊匹配**的关键字，主要用在 `WHERE` 子句里。当你不确定要查的字段**精确等于**什么，只知道一部分内容时，就用 `LIKE`。

它依靠两个**通配符**来干活：

`%` **（百分号）**：代表 **任意数量** 的字符（可以是 0 个、1 个或多个）。

`_` **（下划线）**：代表 **1个** 字符（必须且仅占一个位置）。

在 `CREATE TABLE ... LIKE` 语法中，`LIKE` 的作用是：**创建一个结构完全相同，但不包含任何数据的新表。**

| 复制的内容 | 是否复制 |
| --- | --- |
| 列名、数据类型、长度 | ☑ 完全复制 |
| 主键、唯一索引、普通索引 | ☑ 完全复制 |
| 字段默认值、是否可为空（NOT NULL） | ☑ 完全复制 |
| **表中的数据（行）** | ❌ **不复制**（新表示空表） |
| **外键约束** | ❌ **不复制**（防止表间关联冲突） |
| 自增（AUTO_INCREMENT）的当前值 | ❌ 重置为初始值 |

- 复合索引

`CREATE INDEX idx_state ON customers (lastName,firstName);`

# 保护数据库

- **创建一个用户**

`CREATE USER john@localhost` 本地连接

`CREATE USER john@'%.codewithmosh.com'` john可以替换成别的

`CREATE USER john` 从任何地方都可以连接没有限制

`CREATE USER john IDENTIFIED BY '1234'` 设置一个密码

- **查看用户**

`SELECT * FROM mysql.user`

- **删除用户**

`CREATE USER john@localhost IDENTIFIED BY '1234';`

`DROP USER john@localhost;`

- **更改密码**

更改别人的密码：`SET PASSWORD FOR john = '1234'`

更改自己的密码：`SET PASSWORD = '1234'`

- **授予权限**

网页或者桌面应用

`GRANT SELECT, INSERT, UPDATE, DELETE, EXECUTE ON sql_store.* TO monn_app;` 所有表

`sql_store.customers` 指定表

用户

`GRANT ALL ON *.* TO john` 可以访问所有数据库内的所有表

- **查看权限**

`SHOW GRANT FOR john` 返回john的权限

`SHOW GRANTs` 返回根用户的权限

- **撤销权限**

`REVOKE CREATE VIEW ON sql_store.* FROM moon_app;`